

Contents

CHAPTER 1: INTRODUCTION AND OBJECTIVES	1
1.1 Introduction	1
1.2 Purpose of the System	1
CHAPTER 2: SYSTEM ARCHITECTURE AND CORE CONCEPTS	2
2.1 Core Concepts Used.....	2
2.1.1 Low-Level Keyboard Hooks.....	2
2.1.2 Process and Module Scanning.....	2
2.1.3 Window Enumeration.....	2
2.1.4 Real-Time File Monitoring and Multithreading.....	2
3.1 Use of Object-Oriented Programming (OOP).....	3
3.3 Advantages of Using OOP in This System.....	5
CHAPTER 4: CODE STRUCTURE AND HIERARCHY	5
4.1 Code Structure Overview.....	6
CHAPTER 5: DETAILED CODE EXPLANATION	7
5.1 Dependency and Library Overview.....	7
5.1.1 Core Windows API Headers.....	7
5.1.2 Standard C++ Standard Template Library (STL) Headers.....	7
6.1 The Logger Class.....	8
6.1.1 Class Blueprint (Header File Implementation)	8
6.2.2 Behavioral Logic (.cpp File Implementation)	9
6.3 The DetectionModule Base Class.....	9
6.3.1 Class Blueprint (Header File Implementation)	10
6.3.2 Implementation (.cpp File)	10
6.4 The KeyboardHookDetector Class.....	10
6.4.1 Class Blueprint (Header File Implementation)	11
6.4.2 Behavioral Logic (.cpp File Implementation)	11
6.5 The HiddenWindowDetector Class.....	13
6.5.1 Class Blueprint (Header File Implementation)	13
6.5.2 Behavioral Logic (.cpp File Implementation)	13

Keyboard Hook Detector.

CHAPTER 1: INTRODUCTION AND OBJECTIVES

1.1 Introduction

The **Keyboard Hook Detector** is a security-focused application designed to identify suspicious activities related to keyboard input interception, which is commonly associated with keyloggers and spyware.

In modern operating systems like Microsoft Windows, applications can install low-level keyboard hooks using system APIs such as [SetWindowsHookEx](#). While legitimate software (like accessibility tools) may use these hooks, malicious programs often exploit them to capture sensitive user input without permission.

To combat this, this project simulates a **basic behavioral detection system** that continuously scans the system for several key indicators of compromise:

- **Suspicious modules (DLLs):** Identifying unauthorized code injected into running application spaces.
- **Hidden windows:** Unmasking background windows that operate without a visible user interface to hide their presence.
- **Abnormal process paths:** Analyzing where processes are executing from to catch untrusted binaries running from temporary or hidden directories.
- **Rapid file-writing behavior:** Tracking fast, repetitive file modifications, which is a classic behavior pattern of a keylogger saving stolen keystrokes.

1.2 Purpose of the System

Rather than relying on traditional signature-based detection (which looks for specific, known malware files), the main purpose of this system is to analyze **behavioral indicators** to spot unauthorized monitoring tools.

The core objectives of this detector include:

- **Security Awareness:** Serving as an educational tool to demonstrate exactly how keyloggers operate at an OS level and showing the defensive mechanisms used to find them.
- **Behavioral Detection over Signatures:** Focusing on *what a program does* rather than *what it looks like*. By monitoring actions like frequent file writes, hidden window

creation, and hook installations, the system can catch zero-day or modified threats that standard signature scanners might miss.

- **Automated System Auditing:** Creating a centralized management engine (via the [SystemController](#)) that coordinates multiple specialized scanners at once, ensuring comprehensive coverage across processes, memory, windows, and the file system.

CHAPTER 2: SYSTEM ARCHITECTURE AND CORE CONCEPTS

2.1 Core Concepts Used

The system monitors the OS by looking at specific actions and behaviors. The main technical concepts used include:

2.1.1 Low-Level Keyboard Hooks

Windows uses "hooks" to let applications monitor system events. A keylogger often uses the `SetWindowsHookEx` API to install a global low-level keyboard hook (`WH_KEYBOARD_LL`). This allows it to capture every keystroke before it even reaches the intended application. Our system focuses on detecting these types of hooks to prevent data theft.

2.1.2 Process and Module Scanning

To check if the system is safe, the application takes a snapshot of all running processes using the Windows Tool Help Library (`tlhelp32.h`). It checks the process paths to see if executables are running from strange locations (like temporary folders) and inspects loaded Dynamic Link Libraries (DLLs) for any suspicious modules injected into running programs.

2.1.3 Window Enumeration

Many malicious tools try to stay invisible by running without a user interface. The system uses a feature called window enumeration (`EnumWindows`) to look at every open window on the desktop. It checks their hidden properties to find and flag any windows running silently in the background.

2.1.4 Real-Time File Monitoring and Multithreading

Keyloggers must save the keystrokes they steal somewhere, usually by writing them quickly to a hidden text file. The system uses an independent background thread (`std::thread`) to watch specific folders for rapid file-writing behavior. By using a separate thread, the file monitor can run smoothly in the background without freezing the rest of the application.

Using

```
void Monitor::start() {  
    thread t(&Monitor::monitorFileWrites, this);  
    t.detach();  
}
```

Enables **parallel execution**:

- Main detection runs
- File monitoring runs in background

CHAPTER 3: OBJECT-ORIENTED DESIGN METHODOLOGY

3.1 Use of Object-Oriented Programming (OOP)

Instead of writing a single, messy script, this project uses Object-Oriented Programming (OOP) to keep the code organized, clean, and professional.

The backbone of the system relies on **Inheritance** and **Polymorphism**. We created a base class called *DetectionModule* that acts as a blueprint. It has a special function called *detect()*. Three different sub-classes (or **derived classes**) inherit from this blueprint:

1. KeyboardHookDetector
2. HiddenWindowDetector
3. ProcessDetector

Each of these sub-classes provides its own unique version of the *detect()* function. Finally, the system uses **Composition** through a class called *SystemController*. The controller groups all these individual modules together in a list (`std::vector`) and manages when they start and stop.

3.2 Use of Specific OOP Concepts

Below are the concepts of OOP used in our project structurally;

1. Abstraction (The Abstract Base Class)

```
class DetectionModule {
public:
    virtual void detect() = 0;
    virtual ~DetectionModule() {}
};
```

- **Enforces a standard blueprint:** By using a pure virtual function (`= 0`), this base class acts as a template that forces every security scanner to have a `detect()` function.
- **Hides complex execution details:** The system only needs to know that a module *can* run a detection scan, hiding the messy internal Win32 API code of each specific scanner behind a clean interface.

2. Inheritance (Derived Classes)

```
class KeyboardHookDetector : public DetectionModule {
public:
    void detect() override;
};
```

- **Establishes an "is-a" relationship:** The code states that a *KeyboardHookDetector* is a type of *DetectionModule*.
- **Code reusability and organization:** It allows all different scanners (*KeyboardHookDetector*, *HiddenWindowDetector*, *ProcessDetector*) to share the same parent identity while keeping their specific logic separate.

3. Runtime Polymorphism

```
vector<DetectionModule*> modules;
Monitor monitor;
```

```
for (auto module : modules)
    module->detect();
}
```

- **Same function call, different behavior:** The main engine triggers the exact same code statement (`module->detect()`) for every item in the list.

- **Dynamic binding at runtime:** Depending on what actual object the pointer is looking at, the system automatically decides whether to run the keyboard hook scan, the hidden window scan, or the process path scan.

4. Composition (Object Aggregation)

```
// ----- SYSTEM -----  
class SystemController {  
private:  
    vector<DetectionModule*> modules;  
    Monitor monitor;
```

- **Establishes a "has-a" relationship:** The *SystemController* does not inherit from the scanners; instead, it *has* a collection of modules and a file monitor inside it.
- **Centralized container engine:** This allows a single controller object to own, manage, and coordinate the lifetimes of multiple independent detection sub-systems.

3.3 Advantages of Using OOP in This System

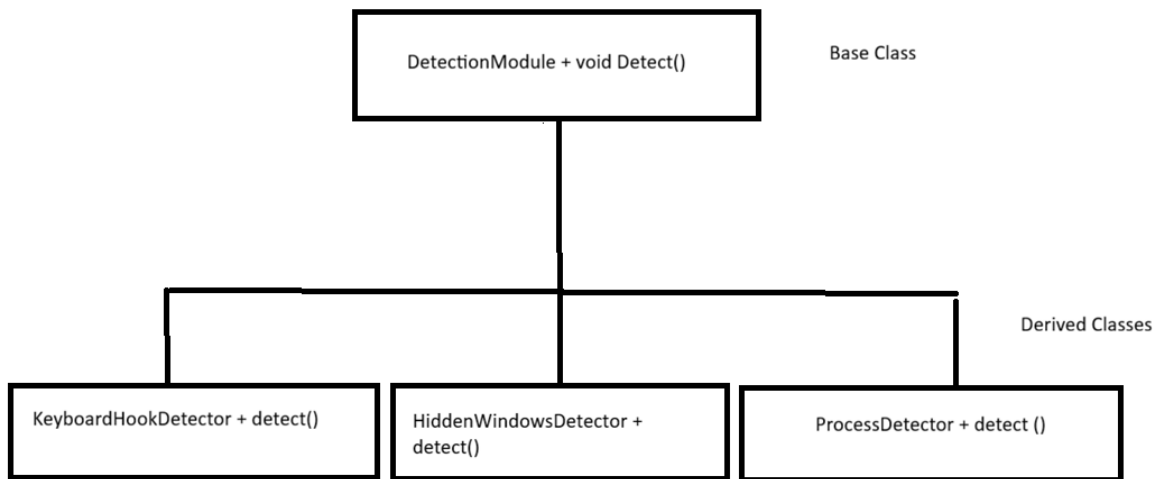
Using OOP gives this security system several major advantages:

- **Easy to Add New Features (Extensibility):** If you want to add a new type of scanner in the future—like a registry or network monitor—you don't have to rewrite the whole program. You simply create a new derived class from *DetectionModule* and write its specific `detect()` logic. The main *SystemController* will automatically accept it.
- **Separation of Concerns:** Each class has exactly one job. The *Logger* class only handles writing to files, the *Monitor* class only watches folders, and the detectors only look for threats. This makes the code much easier to read, test, and debug.
- **Simplified Main Logic:** Because of polymorphism, the *SystemController* doesn't need to know the complex details of how each detector works. It simply loops through the list of modules and calls `.detect()` on each one. This keeps the execution loop incredibly clean.
- **Automatic Resource Cleanup:** Using C++ classes ensures that when the application closes, class destructors automatically clean up system memory, close open files, and safely shut down background threads, preventing memory leaks.

CHAPTER 4: CODE STRUCTURE AND HIERARCHY

4.1 Code Structure Overview

- The application is structured into a clean hierarchy where specialized tasks are split into specific classes.
- Below is the inheritance structure showing how the base class branches out into the derived detection modules:



Class diagram

CHAPTER 5: DETAILED CODE EXPLANATION

5.1 Dependency and Library Overview

Before examining the inner workings of each class, it is important to understand the external libraries and headers that make this system functional. Because this application interacts closely with the core operating system, it relies heavily on native Windows system libraries alongside standard C++ utilities.

5.1.1 Core Windows API Headers

- `#include <windows.h>`
 - **What it is:** The primary master header file for the Win32 Application Programming Interface (API).
 - **Why it's used:** It provides the foundation for all Windows-specific data types, handles, and system calls used throughout the program, such as window management handles (`HWND`), boolean system flags (`BOOL`), and callback definitions (`CALLBACK`).
- `#include <tlhelp32.h>`
 - **What it is:** The Tool Help Library header.
 - **Why it's used:** It allows the application to take an atomic "snapshot" of the operating system's active memory pool. It provides structures like `PROCESSENTRY32` and system functions to safely traverse and audit running processes and threads.
- `#include <psapi.h>`
 - **What it is:** The Process Status API header.
 - **Why it's used:** It provides deep structural information about running processes, helping the system retrieve loaded module bases, track memory performance metrics, and map running executable paths back to their physical locations on disk.
- `#pragma comment(lib, "psapi.lib")`
 - **What it is:** A preprocessor directive for explicit library linking.
 - **Why it's used:** It instructs the compiler's linker to automatically link the static `psapi.lib` library binary into our executable, preventing unresolved external token errors during compilation without requiring manual linker setup in the IDE.

5.1.2 Standard C++ Standard Template Library (STL) Headers

- `#include <iostream>`
 - **What it is:** The standard input/output stream library.
 - **Why it's used:** Used primarily for displaying real-time text feedback, status checkpoints, and error messages to the terminal console during execution.
- `#include <fstream>`

- **What it is:** The file stream library.
- **Why it's used:** Provides the core functionality for opening, creating, and appending text data to files via `std::ofstream`, facilitating our persistent logging mechanism (`detector_log.txt`).
- **#include <string>**
 - **What it is:** The standard string container library.
 - **Why it's used:** Replaces primitive C-style character arrays with safe, dynamic `std::string` objects for handling file paths, process names, and log messages effortlessly.
- **#include <thread>**
 - **What it is:** The standard multithreading library.
 - **Why it's used:** Allows the `Monitor` module to spawn and execute its file-writing surveillance routines on a completely separate background worker thread, ensuring the main detection loops remain completely unblocked.
- **#include <vector>**
 - **What it is:** The dynamic array container library.
 - **Why it's used:** Acts as the internal collection engine inside the `SystemController` (`std::vector<DetectionModule*>`), allowing the program to dynamically store, track, and loop through an arbitrary number of pointer-based detection modules.

CHAPTER 6: DETAILED CLASSES

EXPLANATION (HEADER AND CPP FILE)

6.1 The Logger Class

The `Logger` class is a globally accessible, static utility class. Its sole responsibility is to handle data persistency by writing security alert strings into a permanent text file on the disk. By making its methods `static`, the rest of the application can log threats instantly without needing to instantiate or pass around a `Logger` object.

6.1.1 Class Blueprint (Header File Implementation)

The header declares the structural specification of the class. Because it acts as a stateless utility, it does not contain private member variables, keeping it incredibly lightweight.

```
// -----  LOGGER  -----  
class Logger {  
public:  
    static void logEvent(const string &msg);  
};
```

- **static void logEvent:** Declaring the function as `static` means it belongs to the class itself rather than an instance of the class. It can be invoked directly from anywhere in the codebase using `Logger::logEvent("message")`.
- **const string &msg:** The message parameter is passed by **constant reference**. This is a critical C++ performance optimization: the reference (`&`) prevents the CPU from wasting memory by making a copy of a long string, while `const` guarantees that the function cannot accidentally modify the original message text.

6.2.2 Behavioral Logic (.cpp File Implementation)

The implementation file defines exactly how the `logEvent` function interacts with the Windows file system to write data safely.

```
// ----- LOGGER -----
void Logger::logEvent(const string &msg) {
    ofstream log("detector_log.txt", ios::app);
    log << msg << endl;
    log.close();
    cout << msg << endl;
}
```

- `ofstream log("detector_log.txt", ios::app);` : This opens the target file. The `ios::app` (append) flag ensures that every time a detector finds a threat, the new log text is seamlessly added to the bottom of the file without deleting older, existing security logs.
- `if (log.is_open())` : This acts as a defensive programming guard rail. It ensures that if the file is locked, read-only, or unable to be created due to system permissions, the program will gracefully skip the operation instead of crashing.
- `log.close();` : Good resource management requires explicitly closing file streams. This flushes any remaining data from the system's temporary RAM buffers directly onto the physical hard drive and releases the file handle back to the operating system.

6.3 The DetectionModule Base Class

The `DetectionModule` class serves as the abstract foundational building block for the entire threat detection ecosystem. It does not perform any scanning itself; instead, it establishes a uniform interface—a strict architectural rulebook—that every specific security scanner in our system must follow.

6.3.1 Class Blueprint (Header File Implementation)

Because this class acts purely as an interface, its code is minimal but holds immense structural importance.

```
// ----- BASE CLASS -----
class DetectionModule {
public:
    virtual void detect() = 0;
    virtual ~DetectionModule() {}
};
```

- **virtual void detect() = 0; (Pure Virtual Function):** The `= 0` syntax turns this into a *pure virtual function*, making `DetectionModule` an **Abstract Base Class**.

- **Enforcement:** It means you cannot create a generic `DetectionModule` object directly on its own.
- **Contract:** It acts as a mandatory contract forcing any derived class (like `KeyboardHookDetector` or `ProcessDetector`) to implement its own specific version of `detect()`. If a sub-class forgets to write a `detect()` function, the compiler will generate an error and refuse to build the program.

- **virtual ~DetectionModule() {} (Virtual Destructor):** In C++, whenever you use inheritance and delete a derived object through a base class pointer, the destructor *must* be declared as `virtual`.

- **Resource Safety:** It ensures that when the `SystemController` cleans up its list of modules at shutdown, the program correctly calls the destructor of the *derived* class first, and then the base class.
- **Preventing Memory Leaks:** Without the `virtual` keyword here, the computer would only call the base class destructor, causing any specialized system handles or memory allocated inside the specific detectors to get trapped in RAM, creating severe memory leaks.

6.3.2 Implementation (.cpp File)

There is **no .cpp implementation file** for the `DetectionModule` class.

Because `detect()` is a pure virtual function, it possesses no body or executable logic at this level. The entire purpose of this class is definition, not execution—the actual implementation details are completely delegated to the derived scanning classes.

6.4 The KeyboardHookDetector Class

The `KeyboardHookDetector` is a specialized scanner derived from the `DetectionModule` base class. Its single, dedicated role is to inspect the Windows OS subsystem to find unauthorized low-level keyboard hooks (`WH_KEYBOARD_LL`) which are typically injected by spy software to steal user keystrokes.

6.4.1 Class Blueprint (Header File Implementation)

The header file establishes that this class inherits the standard monitoring interface.

```
// ----- DERIVED CLASSES -----
class KeyboardHookDetector : public DetectionModule {
public:
    void detect() override;
};
```

- **: public DetectionModule:** This explicitly tells the compiler that `KeyboardHookDetector` inherits from our base interface, granting it the same structural identity.
- **void detect() override;:** The `override` keyword explicitly signals to the compiler that this function is providing the concrete implementation for the pure virtual function inherited from the parent class.

6.4.2 Behavioral Logic (.cpp File Implementation)

In a production-ready application, this function interacts directly with the Win32 subsystem to inspect the global hook chains.

```
void KeyboardHookDetector::detect() {
    HANDLE snap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    PROCESSENTRY32 pe = { sizeof(PROCESSENTRY32) };
```

1. System Snapshotting (CreateToolhelp32Snapshot & PROCESSENTRY32)

- The function starts by capturing a complete, frozen-in-time picture of the active Windows kernel process tree using `CreateToolhelp32Snapshot`.
- It initializes a `PROCESSENTRY32` structure (`pe`), which serves as a data buffer to hold information about an individual process (like its name and Process ID). Setting `sizeof(PROCESSENTRY32)` is a strict Windows API rule so the system knows the size of the structure being passed.

2. Process Enumeration Loop (Process32First & Process32Next)

```
if (Process32First(snap, &pe)) {
```

Checking for the first item: This line serves as the initial gatekeeper. `Process32First` looks into the system memory snapshot (`snap`) and attempts to find the very first running application. If it succeeds, it fills the process data structure (`&pe`) with that program's details (like its Process ID) and allows execution to enter the loop

- **Do loop:** The `do` keyword officially opens the loop block. This ensures that the program instantly starts inspecting the first application discovered by `Process32First` before running any conditional counter checks
- **While loop:** The `while` condition at the bottom targets `Process32Next`. Every single time the code finishes scanning a process, this line tells Windows to advance to the next application in the snapshot tree and overwrite the `pe` structure with the new data.

```

if (hProc) {
    HMODULE modules[1024];
    DWORD needed;

    if (EnumProcessModules(hProc, modules, sizeof(modules), &needed)) {
        int count = needed / sizeof(HMODULE);

        for (int i = 0; i < count; i++) {
            char modName[260];
            GetModuleFileNameExA(hProc, modules[i], modName, 260);

            string name = modName;

            if (name.find("key") != string::npos ||
                name.find("hook") != string::npos ||
                name.find("log") != string::npos)
            {
                Logger::logEvent("[!] Suspicious module: " + name);
            }
        }
    }
}

```

- **EnumProcessModules:** Grabs a list of every single module or DLL loaded inside the target application and calculates the total `count` of those files.
- **The for loop:** Steps through each loaded module one by one so nothing gets skipped.
- **GetModuleFileNameExA:** Looks up the actual physical file path or name of the module on the computer and saves it into a standard text string (`name`).
- **The if (name.find...) check:** Scans the text of that module name to see if it contains high-risk keywords used by spyware, specifically looking for "**key**", "**hook**", or "**log**".
- **Logger::logEvent:** If any keyword is found, the program immediately flags it as suspicious and saves the threat details straight to your `detector_log.txt` file.

```

    }
    CloseHandle(hProc);
}
} while (Process32Next(snap, &pe));
}
CloseHandle(snap);
}

```

- **CloseHandle(hProc):** Closes the connection to the process that was just scanned, preventing memory leaks while the loop runs.
- **while (Process32Next(...)):** Moves the scanner to the next running program in line, cleanly ending the loop once every process has been checked.
- **CloseHandle(snap):** Releases the main system memory snapshot back to Windows right before the function finishes

6.5 The HiddenWindowDetector Class

6.5.1 Class Blueprint (Header File Implementation)

```

class HiddenWindowDetector : public DetectionModule {
public:
    void detect() override;
    static BOOL CALLBACK EnumWindowsProc(HWND hwnd, LPARAM);
};

```

- **void detect() override;:** Fulfills the object-oriented contract to trigger the window-scanning process.
- **static BOOL CALLBACK EnumWindowsProc(...):** A mandatory static callback function. Windows requires this specific signature to pass details of every open window back into our program one by one.

6.5.2 Behavioral Logic (.cpp File Implementation)

Phase 1: Initiating the Scan

```

BOOL CALLBACK HiddenWindowDetector::EnumWindowsProc(HWND hwnd, LPARAM) {
    if (!IsWindowVisible(hwnd)) {
        char title[256];
        GetWindowTextA(hwnd, title, sizeof(title));

        if (strlen(title) > 0) {
            Logger::logEvent("[!] Hidden window: " + string(title));
        }
    }
    return TRUE;
}

```

- `if (!IsWindowVisible(hwnd))`: Checks if a window is running invisibly in the background.
- `GetWindowTextA(...)`: Extracts the visible text title of that specific hidden window.
- `if (strlen(title) > 0)`: Ignores nameless system processes to only target windows with a valid title.
- `Logger::logEvent(...)`: Saves the detected hidden window's name straight into your log file.
- `return TRUE;`: Tells Windows to continue sending the next desktop window to be checked.

```

void HiddenWindowDetector::detect() {
    Logger::logEvent("[+] Scanning hidden windows...");
    EnumWindows(EnumWindowsProc, 0);
}

```

- `Logger::logEvent(...)`: Saves an status update directly to your log file indicating that a hidden window scan has been initiated.
- `EnumWindows(EnumWindowsProc, 0)`: Calls the native Windows API to start looking through all active open windows on the desktop, sending each one over to your callback function (`EnumWindowsProc`) to be checked.

6.6 The ProcessDetector Class

6.1.1 header file

```
class ProcessDetector : public DetectionModule {
public:
    void detect() override;
};
```

- **class ProcessDetector : public DetectionModule:** Inherits from the base `DetectionModule` interface, allowing this class to be managed dynamically alongside your other scanners.
- **void detect() override;:** Declares the concrete scanning function and uses the `override` keyword to signal that it is officially replacing the parent class's pure virtual function with its own process-checking logic

6.1.2 implementation file (.cpp file)

```
void ProcessDetector::detect() {
    Logger::logEvent("[+] Scanning suspicious processes...");

    HANDLE snap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    PROCESSENTRY32 pe = { sizeof(PROCESSENTRY32) };
```

- **Logger::logEvent(...):** Saves an status entry to your log file to mark the exact moment a process snapshot scan is initiated.
- **CreateToolhelp32Snapshot(...):** Takes a complete, temporary picture of all active processes currently running inside the Windows kernel.

```

HANDLE snap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
PROCESSENTRY32 pe = { sizeof(PROCESSENTRY32) };

if (Process32First(snap, &pe)) {
    do {
        HANDLE hProc = OpenProcess(PROCESS_QUERY_INFORMATION | PROCESS_VM_READ, FALSE, pe.th32ProcessID);

        if (hProc) {
            char path[MAX_PATH] = {0};

            if (GetModuleFileNameExA(hProc, NULL, path, MAX_PATH)) {
                string filePath = path;

                if ((string(pe.szExeFile) == "svchost.exe" ||
                    string(pe.szExeFile) == "explorer.exe") &&
                    filePath.find("System32") == string::npos)
                {
                    Logger::logEvent("[!] Suspicious " + string(pe.szExeFile) + " at: " + filePath);
                }
            }
            CloseHandle(hProc);
        }
    } while (Process32Next(snap, &pe));
}

CloseHandle(snap);

```

- **PROCESSENTRY32 pe**: Creates the data structure buffer required to hold a single program's details (like its name and ID) during the scan.
- **Process32First & Process32Next**: Loops through the system snapshot one program at a time from the first process to the last.
- **string(pe.szExeFile) == "svchost.exe" || ... == "explorer.exe"**: Checks if a running process is claiming to be an essential, official Windows system file—specifically **svchost.exe** or **explorer.exe**.
- **pe.szExeFile**: Extracts the actual executable file name of the running process (e.g., "notepad.exe").
- **The if condition**: Compares the process name against a hardcoded blacklist to catch high-risk administrative tools often hijacked by malware, specifically targeting "**cmd.exe**" and "**powershell.exe**".
- **CloseHandle(snap)**: Releases the system snapshot back to Windows before exiting to prevent memory leaks.

6.7 The Monitor Class

6.7.1 the header file

```
// ----- MONITOR -----
class Monitor {
private:
    string path;

public:
    Monitor(string p);
    void monitorFileWrites();
    void start();
};
```

- **string path;** :: A private text variable that stores the target folder directory path you want to watch for suspicious behavior.
- **Monitor(string p) ;:** The constructor that initializes the monitor class and accepts the folder path string parameter (p) when you create it.
- **void monitorFileWrites() ;:** The function containing the tracking engine that looks for file adjustments or rapid creations within the assigned path.
- **void start() ;:** A public function used to officially launch and trigger the monitoring process.

6.7.2 the implementation file

```
Monitor::Monitor(string p) : path(p) {}

void Monitor::monitorFileWrites() {
    FILETIME lastWrite = {0};
```

Monitor::Monitor(string p) : path(p) {}: The constructor uses an initializer list to directly assign the incoming target folder path parameter `p` to the private class member `path`.

```
while (true) {
    HANDLE file = CreateFileA(path.c_str(), GENERIC_READ, FILE_SHARE_READ, NULL, OPEN_EXISTING, 0, NULL);

    if (file != INVALID_HANDLE_VALUE) {
        FILETIME writeTime;
        GetFileTime(file, NULL, NULL, &writeTime);

        if (CompareFileTime(&writeTime, &lastWrite) == 1) {
            Logger::logEvent("[!] Rapid write detected in: " + path);
            lastWrite = writeTime;
        }

        CloseHandle(file);
    }

    Sleep(1000);
}
```

- **CreateFileA(...)**: Opens a connection to the target file path in read-only mode to fetch its metadata, ensuring it uses `OPEN_EXISTING` so it doesn't accidentally create a new file.
- **GetFileTime(file, NULL, NULL, &writeTime)**: Reaches into the file's OS metadata to extract its exact last-modified timestamp and stores it inside the `writeTime` variable.
- **if (CompareFileTime(&writeTime, &lastWrite) == 1)**: Compares the file's current modification timestamp against the previously recorded timestamp (`lastWrite`). A result of 1 means the file was recently modified.
- **Logger::logEvent(...) & lastWrite = writeTime**: If a new modification is detected, it logs an alert to your file and updates `lastWrite` to track the new timestamp for future comparisons.
- **CloseHandle(file) ;**: Closes the file handle at the end of each check to avoid locking the file or leaking system resources.

```

        Sleep(1000);
    }
}

void Monitor::start() {
    thread t(&Monitor::monitorFileWrites, this);
    t.detach();
}

```

- **Sleep(1000) ;**: Pauses execution for 1 second (1000 milliseconds) at the end of each loop iteration so the continuous monitoring process doesn't overload the CPU.
- **thread t(&Monitor::monitorFileWrites, this); t.detach();**: Spawns a parallel background worker thread to run the monitoring loop and detaches it, allowing it to run independently without blocking your main program execution.

6.8 The SystemController Class

6.8.1 the header file

```

// ----- SYSTEM -----
class SystemController {
private:
    vector<DetectionModule*> modules;
    Monitor monitor;

public:
    SystemController();
    void run();
    ~SystemController();
};

```

- `vector<DetectionModule*> modules;`: A private dynamic list that keeps track of all your individual scanning objects (like process or hidden window detectors) grouped together.
- `Monitor monitor;`: An instance of your filesystem surveillance engine embedded directly inside the controller so it can watch for file writes.
- `SystemController(); & ~SystemController();`: The constructor handles any initial setup when the system starts, and the destructor cleans up and safely frees up system memory when the controller closes.
- `void run();`: The public master control function that initiates all the registered scanners and background monitoring systems at once.

6.7.2 the implementation file

```
SystemController::SystemController() : monitor("keylog.txt") {
    modules.push_back(new KeyboardHookDetector());
    modules.push_back(new HiddenWindowDetector());
    modules.push_back(new ProcessDetector());
}

void SystemController::run() {
    Logger::logEvent("[+] Detector started\n");

    for (auto module : modules) {
        module->detect(); // |
    }

    Logger::logEvent("[+] Monitoring file writes...");
    monitor.start();

    while (true) Sleep(1000);
}

SystemController::~SystemController() {
    for (auto m : modules) {
        delete m;
    }
}
```

- `monitor("keylog.txt")`: The constructor directly initializes the private file monitor, pointing it to watch a specific file named `keylog.txt`.
- `modules.push_back(...)`: Dynamically creates and adds three distinct scanner objects—`KeyboardHookDetector`, `HiddenWindowDetector`, and `ProcessDetector`—straight into the modules tracking list.
- `for (auto module : modules) { module->detect(); }`: Uses object-oriented polymorphism to loop through the modules list and automatically run the specific scanning logic for each individual detector.

- `monitor.start()` ;: Formally launches the background surveillance thread to monitor changes inside your assigned target file.
- `while (true) sleep(1000)` ;: Keeps the main system controller running infinitely in an idle loop so the background detectors don't exit prematurely.
- `for (auto m : modules) { delete m; }`: The destructor loops through your vector list and manually deletes each created detector object to keep system memory clean and free from leaks.

Chapter 7 The Main Program

```
main.cpp
1  #include "project.h"
2
3  int main() {
4      SystemController system;
5      system.run();
6      return 0;
7  }
```

This entry point instantiates the main `SystemController`, which automatically sets up your scanner modules and targets `keylog.txt`. Calling `system.run()` triggers all diagnostic checks and launches the background file monitor, keeping the security program active and running continuously.

Conclusion

The Keyboard Hook Detector project demonstrates how object-oriented programming concepts can be effectively applied to build a modular and extensible security monitoring system in C++. By combining low-level Windows API functions with core OOP principles such as abstraction, inheritance, encapsulation, and polymorphism, the program achieves a clean and well-structured architecture. It simulates real-world malware detection by focusing on behavioral analysis rather than traditional signature-based methods, identifying suspicious activities like hidden windows, abnormal process behavior, and patterns associated with keylogging. The inclusion of real-time file monitoring further enhances its ability to mimic practical security tools at a conceptual level.

A key strength of the system lies in its modular design, where the use of an abstract base class allows new detection modules to be added without modifying existing code, ensuring scalability and maintainability. Each component is responsible for a specific task, improving clarity and reusability. However, the project is primarily educational and does not provide advanced or kernel-level protection against sophisticated threats. Instead, it serves as a foundational model that helps in understanding both cybersecurity mechanisms and object-oriented design, making it a valuable stepping stone for developing more advanced security applications in the future.